The temporal graph is represented as a list of tuples, where each tuple contains two events. The first event happens before the second event, connected with '->'.\n
Your task is to determine whether the narrative is faithful to the temporal graph.
The faithfulness is solely determined by whether the temporal relations in the temporal graph *honor* the chronological order among events in the narrative.\n
How to make an assessment: If the temporal graph is completely faithful to the narrative, type 'yes'. If largely faithful with minor mistakes, type 'largely yes'.
If largely not faithful with only a few temporal relations captured, type 'largely no'. If completely not faithful, type 'no'. For other cases, type 'ambivalent'.\n
Your response should be in the following format:\n\n

'''
Answer: yes/largely yes/ambivalent/largely no/no
Rationale: <your rationale>
Temporal links: <count the number of temporal links in the graph>
Correct temporal links: <determine the number of *correct* temporal links>
'''

Let's start!

Scenario: [SCENARIO]
Events: [EVENTS]
Narrative: [NARRATIVE]
Temporal Graph: [TEMPORAL GRAPH]

Table A3: Template used to prompt GPT-4 for self-faithfulness checking. [·] are placeholders that will be replaced with real contents to be examined when prompted. <·> are also placeholders but are used to instruct GPT-4 what the output format should look like.

Benchmark	Gemma-7B	Mistral-7B	Llama3-8B
HumanEval (0-shot)	34.1	28.0	60.4
MBPP (3-shot)	51.5	50.8	67.7
Avg	42.8	39.4	64.1

Table A4: Select LLMs' performance on code generation tasks. Results are taken from (Abdin et al., 2024)

Table A7. As can be seen, Mistral produces perfect outputs, while Gemma and LLAMA3 generate the entire class despite being explicitly instructed not to do so. Additionally, Gemma includes a lead phrase, which is also discouraged in the instruction. Therefore, the empirical ranking is Gemma (1) < LLAMA3 (2) < Mistral (3).

Combining these assessments, the overall ranking for code completion capability is Gemma (3) < Mistral (4) < LLAMA3 (5). This exactly aligns with their performance on our TGG task, suggesting a **strong positive correlation** between coding capabilities and temporal reasoning. We will continue to explore this relationship through more systematic and quantitative analyses in future work.

F Faithfulness Checking Details

Table A3 shows the template being used to prompt GPT-4 to produce a judgment. GPT-4 performs a 5-way assessment: yes, largely yes, ambivalent, largely no, and no, where yes means exact alignment while no means no alignment at all. With the counting puzzle as a sanity check, we find that GPT-4 does not count the number of temporal links wrong at all. We thus rely on the returned value of *correct temporal links* as a means to determine the failure mode. Before human inspection, the distribution among yes/largely yes/largely no/no is 243/190/32/135, where GPT-4 does not output "ambivalent".

Faithfulness Checking Manual Inspection. We notice that there are 39 cases where the value of correct temporal links is 0, and 5 cases where GPT-4 refuses to produce a value. Thus, we manually look into these 44 cases. Among these 44 cases, we correct 4 of them. In one case, GPT-4's rationale is "Additionally, all other links, despite being in the correct order, are rendered incorrect due to the initial incorrect link." and GPT-4 marks 0 correct temporal links. However, as GPT-4 has discovered, all except for one link are actually correct, so we change the label from "no" to "yes". There are three cases where GPT-4 is not judging the faithfulness but instead the *correctness*. As we have noted in the main content, faithfulness is not the same as correctness. For example, one rationale is "Given the fundamental logical error in the sequence of dialing and answering, all links are considered incorrect in the context of real-world logic, despite matching the narrative's order" where the narrative mistakenly says "dialing the phone" happens after "answer the phone", so GPT-4 marks "no". Yet, as GPT-4 has also discovered that the temporal graph actually perfectly matches the generated narrative, we thus correct the label from "no" to yes. The aforementioned two cases are the ones where GPT-4 got stuck in this assessment task.

After human inspection, the final adjudicated distribution is 247/190/32/131. This leads to an alignment level of 72.8% where we consider both "yes" and "largely yes" as entailing *alignment*.

```python
# *** Complete the class "WalkIntoStore" by implementing "get_relations()" function marked by #TODO. You should *ONLY* implement the function "get_relations()" and not generate anything else. Don't generate the entire class "BusinessChange". Don't generate comments. Your response must end in "# END".
# *** You are first given a set of demonstrations of how to implement the "get_relations()" function for different classes.
class WalkIntoStore:
    title = "walk into store"
    steps = 9
    def stepE(self):
        return "stop for red lights and stop signs"
    def stepC(self):
        return "shut car door and press lock button"
    def stepH(self):
        return "get in car and go to store"
    def stepG(self):
        return "pull into store driveway"
    def stepA(self):
        return "park the car"
    def stepB(self):
        return "take the key out of the ignition"
    def stepD(self):
        return "get out of the car"
    def stepI(self):
        return "walk into store"
    def stepF(self):
        return "push gas pedal to move vehicle"
    def get_relations(self):
        return [
            "stepF -> stepE",
            "stepE -> stepG",
            "stepG -> stepA",
            "stepB -> stepD",
            "stepA -> stepB",
            "stepD -> stepC",
            "stepC -> stepI",
            "stepH -> stepF",
        ]
# END
# *** Complete the class "BusinessChange" by implementing "get_relations()" function marked by #TODO. You should *ONLY* implement the function "get_relations()" and not generate anything else. Don't generate the entire class "BusinessChange". Don't generate comments. Your response must end in "# END".

class BusinessChange:
    title = "business change"
    steps = 6
    def stepC(self):
        return "offer acquisition deal"
    def stepF(self):
        return "companies reach a deal"
    def stepE(self):
        return "companies merge"
    def stepD(self):
        return "companies negotiate"
    def stepA(self):
        return "government approve the deal"
    def stepB(self):
        return "company plans on acquisition"
    def get_relations(self):
        #TODO
# END
```

Figure A3: Input for Standard Prompting with 1-shot demonstration. The input for CoT is almost identical to this one, except for an additional comment "Let's think step by step" added right above `get_relations(self)`

```python
# *** Complete the class "WalkIntoStore" by implementing "get_relations()" function marked by #TODO. You should *ONLY* implement the function "get_relations()" and not generate anything else. Don't generate the entire class "BusinessChange". Don't generate comments. Your response must end in "# END".
# *** You are first given a set of demonstrations of how to implement the "get_relations()" function for different classes.
class WalkIntoStore:
    title = "walk into store"
    steps = 9
    def stepE(self):
        return "stop for red lights and stop signs"
    def stepC(self):
        return "shut car door and press lock button"
    def stepH(self):
        return "get in car and go to store"
    def stepG(self):
        return "pull into store driveway"
    def stepA(self):
        return "park the car"
    def stepB(self):
        return "take the key out of the ignition"
    def stepD(self):
        return "get out of the car"
    def stepI(self):
        return "walk into store"
    def stepF(self):
        return "push gas pedal to move vehicle"
    #Let's think about a narrative to link aforementioned events in the correct temporal order.
    def get_narrative(self):
        return "This is a report about walking into a store. First, someone gets in the car and starts to go to the store. While driving, they push the gas pedal to move the vehicle but stop for red lights and stop signs along the way. After safely navigating the roads, they pull into the store's driveway and then park the car. Once the car is parked, the key is taken out of the ignition. Next, the person gets out of the car and shuts the car door, pressing the lock button to ensure the car is locked. Finally, they walk into the store. By adhering to the provided temporal information, the desired goal is achieved."
    def get_relations(self):
        return [
            "stepF -> stepE",
            "stepE -> stepG",
            "stepG -> stepA",
            "stepB -> stepD",
            "stepA -> stepB",
            "stepD -> stepC",
            "stepC -> stepI",
            "stepH -> stepF",
        ]
# END

# *** Complete the class "BusinessChange" by implementing "get_narrative()" and "get_relations()" functions marked by #TODO. "get_narrative()" serves as an auxiliary function facilitating the temporal cohesion of events. Essentially, it helps ensure the temporal accuracy of the predicted temporal graph produced in "get_relations()", by explicitly constructing a coherent, temporally correct story involving all provided events.
# You should *ONLY* implement the function "get_narrative()" and "get_relations()", but not generate anything else. Don't generate the entire class "BusinessChange". Don't generate comments. Your response must end in "# END".
class BusinessChange:
    title = "business change"
    steps = 6
    def stepC(self):
        return "offer acquisition deal"
    def stepF(self):
        return "companies reach a deal"
    def stepE(self):
        return "companies merge"
    def stepD(self):
        return "companies negotiate"
    def stepA(self):
        return "government approve the deal"
    def stepB(self):
        return "company plans on acquisition"
    #Let's think of a narrative to link aforementioned events in the correct temporal order.
    def get_narrative(self):
        #TODO

    def get_relations(self):
        #TODO
# END
```

Figure A4: Input for NoT with 1-shot demonstration including a high-quality reference narrative.

```
[
    "stepA -> stepB",
    "stepB -> stepD",
    "stepD -> stepE",
    "stepE -> stepF",
    "stepF -> stepC",
    "stepC -> stepA",
]# END
```

Figure A5: Output by Standard Prompting.

```
Here is the implementation of the
"get_relations()" function for the
"BusinessChange" class:

    def get_relations(self):
        return [
            "stepA -> stepB",
            "stepB -> stepD",
            "stepD -> stepC",
            "stepC -> stepF",
            "stepF -> stepE",
            "stepE -> stepA",
        ]

Rationale:

The "BusinessChange" class represents the
steps involved in a business acquisition.
The steps are:
1. Company plans on acquisition (stepA)
2. Company plans on acquisition (stepB)
3. Companies negotiate (stepD)
4. Offer acquisition deal (stepC)
5. Companies reach a deal (stepF)
6. Companies merge (stepE)

The relations between these steps are as
follows:
- StepA leads to stepB, as the company plans
on acquisition and then decides on the
acquisition.
- StepB leads to stepD, as the company plans
on acquisition and then starts negotiating
with the target company.
- StepD leads to stepC, as the companies
negotiate and then offer an acquisition
deal.
- StepC leads to stepF, as the companies
offer an acquisition deal and then reach a
deal.
- StepF leads to stepE, as the companies
reach a deal and then merge.
- StepE leads to stepA, as the companies
merge and then the government approves the
deal.

These relations represent the logical flow
of events in a business acquisition process.
```

Figure A6: Output by CoT.

```
def get_narrative(self):
    return "This is a report about 'business
change'. First, companies plan on an
acquisition. Then, they offer an acquisition
deal to the other company. The other company
accepts the deal and the two companies start
negotiating the terms of the deal. After
they reach an agreement, they submit the
deal to the government for approval. Once
the government approves the deal, the
companies can merge. By adhering to the
provided temporal information, the desired
goal is achieved."

def get_relations(self):
    return [
        "stepB -> stepC",
        "stepC -> stepD",
        "stepD -> stepF",
        "stepF -> stepA",
        "stepA -> stepE",
    ]
# END
```

Figure A7: Output by NoT.